Output Parsers

LLMs return output as unstructured text (strings), but applications (Databases, APIs, Frontends) need structured data. Parsers acts as the 'Translator' layer. They take the raw text output from the LLM, strip away metadata, and convert it into a suitable format for other applications.

(A) String Output Parser: It extracts the actual content from the LLM response object, stripping away token usage counts and metadata

(B) JSON Output Parser: It forces the LLM to return outputs in valid JSON format. It has a consequence that it doesn't enforce any schema meaning it can ensure that we recieve some JSON, but it doesn't guarantee specific keys (e.g. we might want {'age': 10} but get {'years': 10} in the output.

(C) Structured Output Parser: It uses ResponseSchema to tell LLM exactly what keys (fields) we need in the JSON. It enforces the structure keys (e.g. strictly requires name, age, city).
It also has a cons that there is no data validation. It cannot ensure data types. If the LLM returns {'age': 'twenty'}, this parser accepts it, even if our application needs integer.

(D) Pydantic Output Parser: It uses Python's Pydantic library to define a data class. It provides all the benefits of the standard Pydantic library like:

→ schema enforcement ⇒ Guarantees Keys
→ data validation ⇒ Checks data types (eg. ensures 'age' is 'int')
→ logic checks ⇒ can enforce rules (e.g. age>18). If the LLM fails, it throws an error.

It is the gold standard and should be used in production apps where data integrity is critical.

Scanned with OKEN Scanner

Chains

Chains are wrappers that connect multiple components (Prompts, Models, Parsers) into a single pipeline.

Using chains directly makes output of step 1 become input for step 2 automatically. We don't need to manually store variables and pass them to the next function thus avoiding redundancy.

It allows us to build sophisticated workflows (like Parallel processing or branching logic) rather than writing messy code.

LangChain Expression Language (LCEL)

Standard Pattern: Chain = Prompt | Model | Parser

We only have to invoke the chain once: chain.invoke(input)

(a) Sequential Chain (Linear) Here, the execution happens step-by-step. A → B → C. Example: User inputs a topic (eg. Unemployment) → LLM1 generates a detailed report → LLM2 takes that report and summarizes it into 5 bullet points.

This replaces the need to manually copy-paste the output of the first model into the second.

(b) Parallel Chain Using this we can split one input into multiple simultaneous processes. We use the **Runnable Parallel** component present in Lang Chain. Eg: We can provide a long text paragraph/topic.

Branch 1 (Model A): Generates study notes

Branch 2 (Model B): Generates a quiz.

Merge: A third step combines both outputs into a final document.

It helps to reduce latency by running independent tasks at the same time.

(c) Conditional Chain (Routing) · We can perform conditional logic (If/Else) to decide which path to take. **Runnable Branch** component needs to be used for this. Example: Sentiment Analysis.

Input: User feedback

Classifier: Determines if feedback is positive or negative.

Routing: ⇒ If positive ⇒ Trigger "Thank You" chain

⇒ If negative ⇒ Trigger "Apology / Support" chain.

Scanned with OKEN Scanner